

CS151 Data Structures

Quiz: 1

Date: 9/10/26

Name:

```
1.  try {
        int x = 10/0;
        String s = null;
        System.out.println(s.length());
    }
    catch (ArithmeticException e) {
        System.out.println("Arithmetic error");
    }
    catch (NullPointerException e) {
        System.out.println("Null pointer error");
    }
}
```

```
System.out.println("Program continues");
```

- (a) Which exception is thrown?
- (b) What is the output of the program?
- (c) Is the `NullPointerException` code block ever executed? If not, why not?
- (d) If your answered no above, modify the program so that the output will indeed be

```
Arithmetic error
Null pointer error
Program continues
```

2. Write a Java program to create a custom exception called `InvalidScoreException`. The program should read a floating point number from input and throws the exception if the number is below 0 or above 100. Catch the exception and print an appropriate message.

3. Given the following class definitions:

```
class A {
    void calc(int n) {
        n *= 3;
        println(n);
    }
}
class B extends A {
    void calc(int n) {
        n += 2;
        super.calc(n);
        println(n);
    }
}
```

What is printed by the following code segment?

```
A x = new B();
x.calc(5);
```

4. Given the following class definition

```
class Product {
    float size; float weight;
    //initializes the instance variables size and weight appropriately
    //based on the incoming parameters
    Product(float size, float weight) {
        //code not shown
    }
    //returns true if this product is considered defective based on some
    //specific computation involving size and weight
    boolean isDefective() {
        //code not shown
    }
}
```

(a) Write a class **EnhancedProduct** that extends **Product**, with the following requirements

- i. One additional instance variable **smoothness**, of type **int**
- ii. A constructor that takes three parameters, **size**, **weight** and **smoothness** and uses them to initialize all instance variables of **EnhancedProduct** appropriately.
- iii. Override the method **isDefective** in **EnhancedProduct** so that it returns **false** only if it is not defective based on **Product**'s rules and **smoothness** is greater than 5, and **true** otherwise

(b) Write a method that takes an array of **Product** (which contains a mix of **Product** and **EnhancedProduct**) and returns the total number of defective items in the array.